Database Indexing

ISYS2099 | ISYS3434 · Database Applications · Week 2

Tom Huynh
School of Science, Engineering & Technology
RMIT University Vietnam



RMIT UNIVERSITY

Today's plan

01

Apps & performance

How database apps are built and how we measure speed

≈ 12 min

02

Inside MySQL storage

Storage engines, InnoDB, pages and tablespaces

≈ 12 min

03

Why indexes?

Disk access is slow: avoid reading every row

≈ 7 min

04

How indexes work

Primary, secondary, multilevel, B+-trees, composite, hash

≈ 27 min

05

Using indexes well

SQL, EXPLAIN, guidelines, FULLTEXT and spatial

≈ 19 min

Then the tutorial (≈ 5 min briefing): build a 100,000-row products table, time five queries with and without indexes, then try FULLTEXT search.



01

Database Apps & Performance

Part 01

What a database app is

Three tiers

Performance metrics

RMIT UNIVERSITY

What is a database application?

A database application is a program whose main job is entering and retrieving information from a database.



Desktop era

Before the mid-1990s

Airline reservation systems: desktop programs talking to a central database.



Web era

From the mid-1990s

Web interfaces become normal. amazon.com ran on an Oracle back-end database.

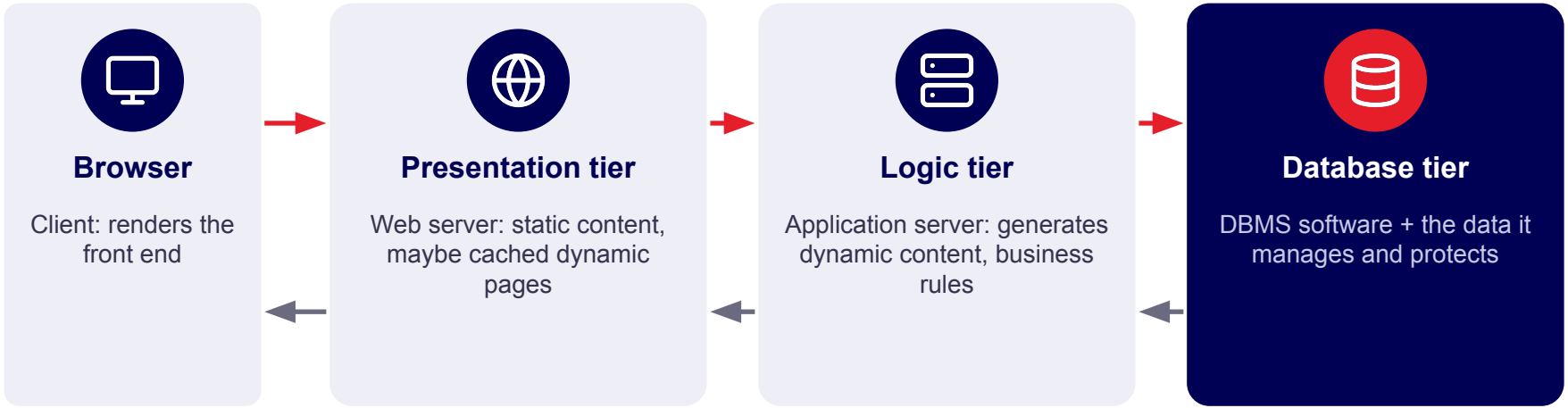


Planet scale

Today

Facebook still runs on MySQL, plus caching layers such as Memcached, to serve 3+ billion monthly users (2026).

Web database applications: three tiers



→ request ← response

Cache layer (e.g. Memcached) keeps hot data in memory between app servers and MySQL

Your group project is a web database application. This week we make its database tier fast.

Four ways to measure database performance



Throughput

How much is the database doing?

requests or transactions per second; orders taken per second



Execution time

How long does each job take?

milliseconds per query, not just how many queries



Concurrency

How many jobs at the same time?

more concurrent tasks change how resources are shared and add waiting



Utilization

What % of the time is it busy?

= throughput × execution time; near 100% means new requests start to queue

Your turn: which coffee shop would you build?

A **10 machines**



each makes 1 cup per minute

B **1 machine**



makes 5 cups per minute (12 s per cup)

Both options cost the same.

On your own · 3 min Compare throughput and execution time. Which option is better if 5, 10 or 20 customers arrive per minute? Post your three answers in the chat.

Coffee shop answer: it depends on the load

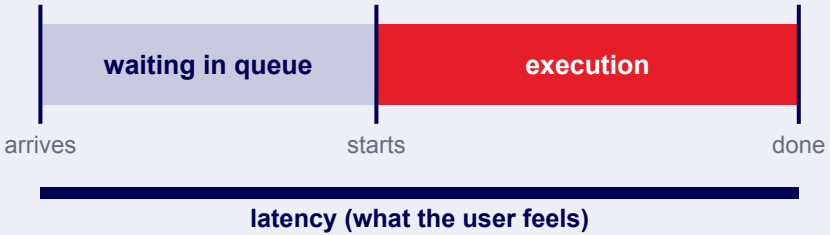
	Option A (10 machines)	Option B (1 fast machine)
Throughput	10 cups / min	5 cups / min
Execution time	60 s per cup	12 s per cup
5 customers / min	Everyone waits 60 s	Everyone waits 12 s ✔
10 customers / min	Keeps up: 60 s each ✔	Overloaded: queue keeps growing
20 customers / min	Overloaded, queue grows slowly ✔	Overloaded, queue grows fast



Low execution time wins at low load; high throughput wins at high load. Latency = waiting time + execution time.

Latency, concurrency and utilization

Latency = waiting + execution

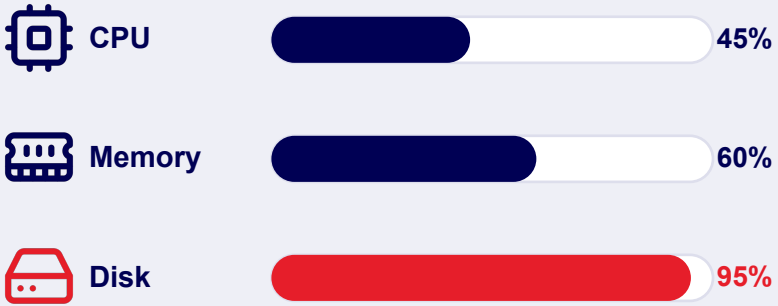


More concurrent requests → longer queues



The new request (red) waits for the four in front of it.

Utilization: how busy is each resource?



example values

Critical resource: overused, or out of proportion to the others. Here the disk is the bottleneck.

Three tools to make databases fast



Indexing

Week 2 · today

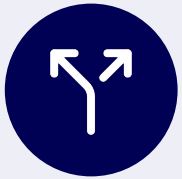
Find matching rows without reading the whole table.



Query optimization

Week 3

Write queries the optimizer can run efficiently, and read the plan it chooses.



Partitioning

Week 4

Split a very large table into smaller pieces that are easier to search and manage.

All three attack the same enemy: unnecessary disk access.



02

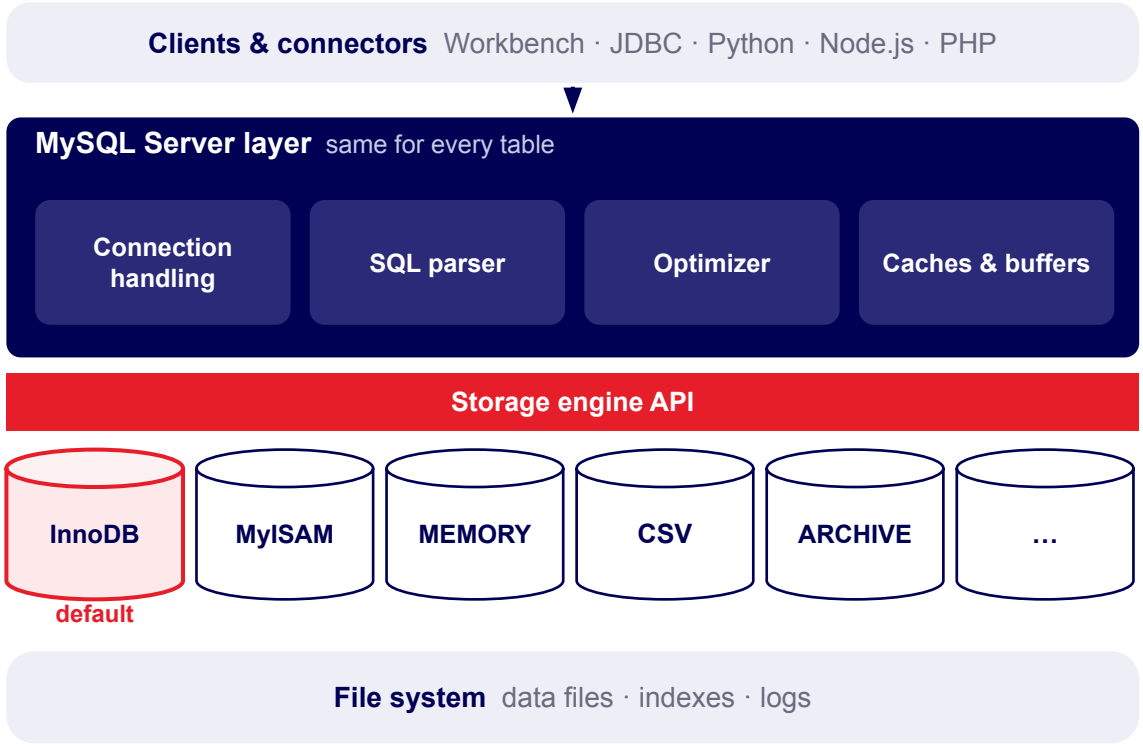
Inside MySQL Storage

Part 02

Storage engines | InnoDB | Pages & extents | Tablespaces

RMIT UNIVERSITY

MySQL's pluggable storage engine architecture



Plug-in design

The server layer parses and optimizes every query the same way.

The storage engine decides how a table's data is stored, locked and indexed.

Chosen per table:

```
SHOW ENGINES;

CREATE TABLE t (...)
  ENGINE = InnoDB;
```

Choosing an engine changes how a table behaves

Affects	InnoDB (default)	MyISAM	MEMORY
Concurrency	Row-level locking	Table-level locking	Table-level locking
Transactions	Yes, ACID	No	No
Referential integrity	Foreign keys enforced	No foreign keys	No foreign keys
Crash recovery	Automatic	Limited (repair tables)	Data lost on restart
Physical storage	.ibd tablespace files	.MYD data + .MYI index	RAM only
Indexing support	B-tree, FULLTEXT, SPATIAL	B-tree, FULLTEXT, SPATIAL	HASH (default), B-tree
Memory caches	Buffer pool: data + indexes	Key cache: indexes only	Everything in memory

Performance aids differ too, e.g. InnoDB's change buffer and adaptive hash index. **This course uses InnoDB.**

InnoDB: the default engine since MySQL 5.5

A general-purpose engine that balances high reliability and high performance.



ACID transactions

All-or-nothing changes that survive crashes (Week 6).



Row-level locking

Many users can update different rows at the same time.



Primary & secondary indexes

Rows are stored in primary key order: the clustered index.



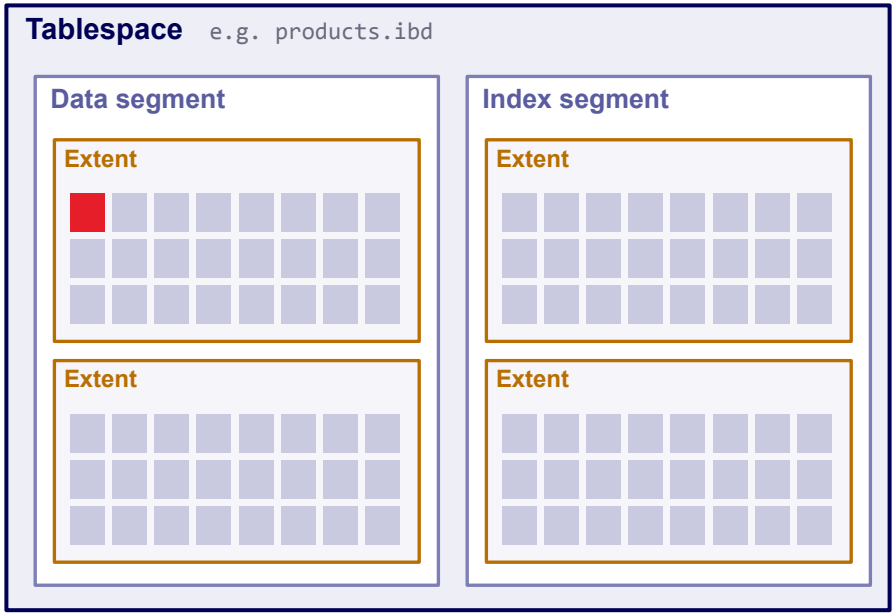
Foreign key constraints

Enforces referential integrity between tables.

Check a table's engine: `SHOW CREATE TABLE products;`

How InnoDB organises storage

Logical structure (what the DBMS manages) is separate from **physical structure** (files on disk), so storage can change without changing tables or queries.



Tablespace
One or more files holding the data of one or more tables and their indexes.

Segment
A section of a tablespace, e.g. one for data and one for an index. Grows and shrinks.

Extent
A group of contiguous pages. InnoDB: 64 pages × 16 KB = 1 MB.

Page
The unit moved between disk and memory (InnoDB default 16 KB). Holds one or more rows.

Block
Textbook word for a page: the unit read from disk. Its size is a multiple of the OS block (often 4 KB).

Why tablespaces? General benefits

Allocate space

Easier to set space quotas for different users.

Back up by unit

Partial backups and recoveries, one tablespace at a time.

Spread the I/O

A large object (e.g. a partitioned table) can span several disks and controllers.

Work on one part

Take a tablespace offline without stopping the whole database.

Move data

Import or export application data one tablespace at a time.

Exact features vary by DBMS; some of these come from enterprise systems such as Oracle.

Types of InnoDB tablespaces



System tablespace

ibdata1

Change buffer and internal data; holds user tables only if innodb_file_per_table = OFF. (MySQL 8: the data dictionary is in mysql.ibd.)



File-per-table

default

One .ibd file per table holding its data and indexes. Used automatically by CREATE TABLE.



General tablespace

shared

One tablespace for many tables. You create it with CREATE TABLESPACE.



Undo & temporary

internal

Undo logs for rolling back changes; temporary tables (#innodb_temp).

File-per-table tablespaces: pros and cons

Advantages

Space goes back to the OS
after TRUNCATE or DROP TABLE

Faster TRUNCATE TABLE
than tables in a shared tablespace

Files can live on separate devices
for I/O optimization, space management or backup

Each table can grow to 64 TB
tables in a shared tablespace share one 64 TB limit

Disadvantages

Unused space inside a table's file can only be reused by rows of that same table

so space can be wasted if it isn't managed.

Many tables = many files

one open file handle and separate fsync calls per table.

Default setting:

```
innodb_file_per_table = ON
```

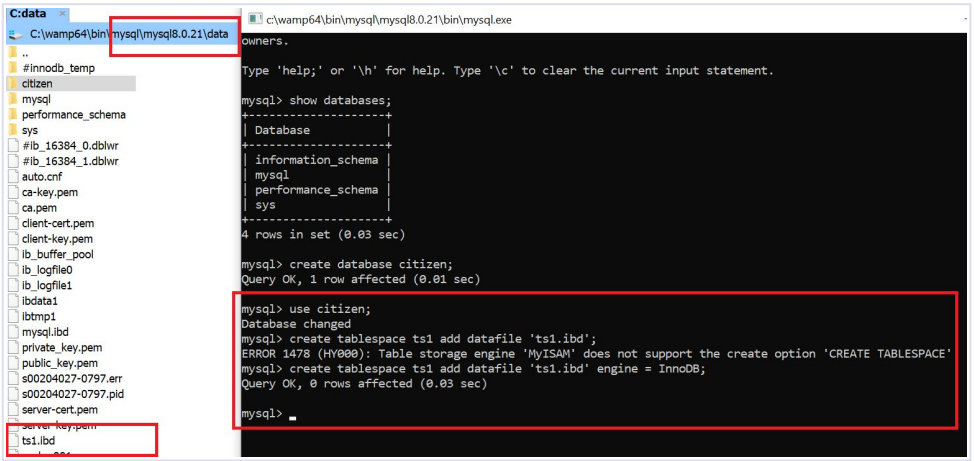
Creating and using a general tablespace

Create it, then place tables in it

```
CREATE TABLESPACE ts1
  ADD DATAFILE 'ts1.ibd'
  ENGINE = InnoDB;

-- new table in ts1
CREATE TABLE t1 (c1 INT PRIMARY KEY)
  TABLESPACE ts1;

-- move an existing table
ALTER TABLE t2 TABLESPACE ts1;
```



Lesson from this screenshot: the server's default engine was MyISAM, so CREATE TABLESPACE failed until **`ENGINE = InnoDB`** was added. ts1.ibd then appears in the data directory.



03

Why Indexes?

Part 03

Data keeps growing

Memory hierarchy

Scan vs search

RMIT UNIVERSITY

Why database optimization?



Data keeps growing

As data grows, queries slow down. Growth is often faster than linear: think of Facebook users or the Amazon catalogue.



Hardware lags behind

Don't expect disk speeds to grow exponentially over the next decade. Buying faster hardware is not enough.

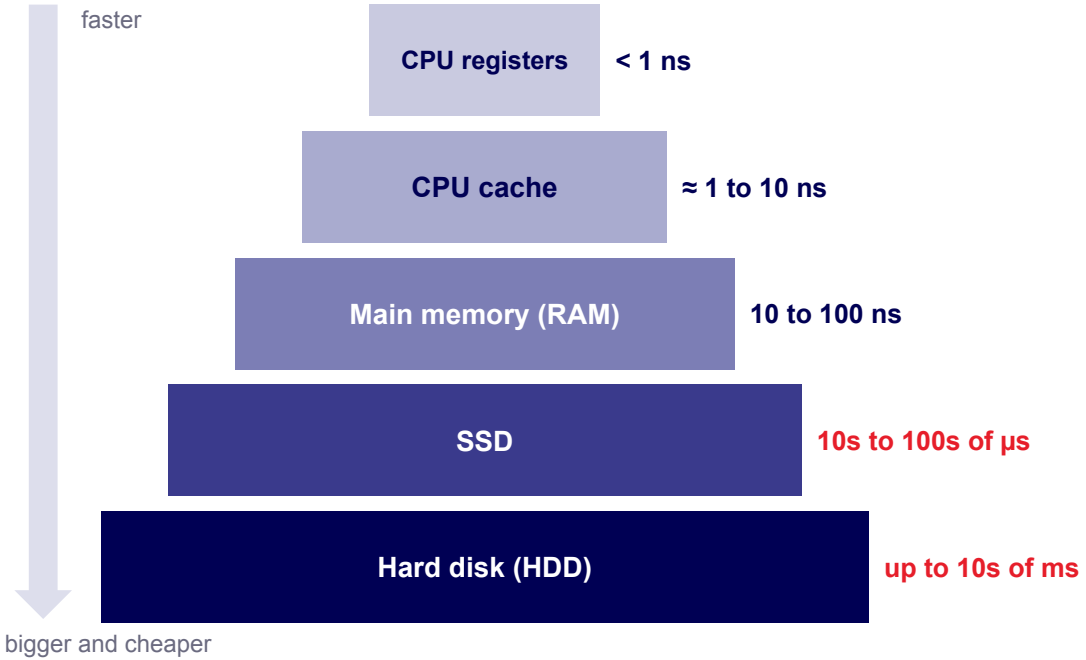


Disk access dominates

Most of a slow query's time is spent waiting for data to come off disk.

Rule #1 of database performance: disk access is the most expensive thing a database does.

The memory hierarchy: why disk is so slow



Latencies are typical orders of magnitude. Source: en.wikipedia.org/wiki/Memory_hierarchy

5 to 6 orders of magnitude between RAM and a hard disk. Even an SSD is about 1,000× slower than RAM.

Feel it: if reading RAM took 1 second, reading a hard disk would take more than a day.

Disk I/O is unavoidable: data doesn't fit in memory and must survive shutdowns and crashes. So databases work hard to read as few blocks as possible.

Guess: how many rows must MySQL read?

```
SELECT * FROM people WHERE id = 123456;
```

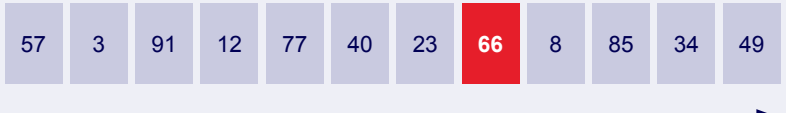
The table has **5 million rows** and no index. Type 1, 2, 3 or 4 in the chat.

1	2	3	4
5,000,000	**About 2,500,000**	**About 23**	**Exactly 1**

Then think: what would have to be true for the answer to be 23?

Two ways to find id = 123456

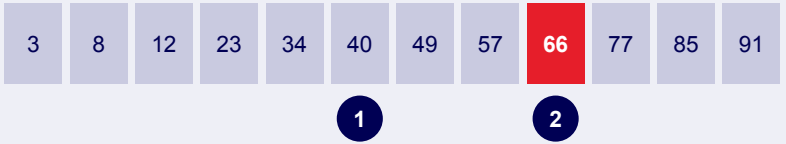
1 Full table scan



10 rows: fine, only 10 reads.

5 million rows: up to 5 million reads, every time.

2 Sorted copy + binary search



5 million rows: $\lceil \log_2 5{,}000{,}000 \rceil = 23$ lookups.

But: sorting 5 million rows is expensive, and the copy is sorted on one column only.

The idea: keep a small, sorted, auxiliary structure next to the table that points to the rows. That is an **index**.

04

How Indexes Work

Part 04

Index entries | Primary & secondary | Multilevel | B-tree & B+-tree | Composite & hash

RMIT UNIVERSITY

How an index works

INDEX sorted by Data

Data	Location
GLASS	6
JONES	2
JONES	9
PLEW	5
SMITH	1
SMITH	3
SMITH	7
SMITH	100,000
WALLACE	8
WILLIAMS	4

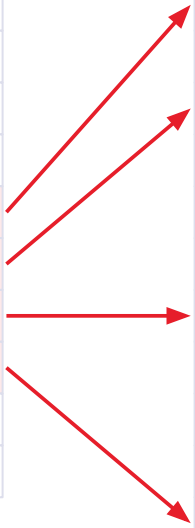


TABLE rows as inserted

Location	Data
1	**SMITH**
2	JONES
3	**SMITH**
4	WILLIAMS
5	PLEW
6	GLASS
7	**SMITH**
8	WALLACE
9	JONES
…	…
100,000	**SMITH**

Index entry = `<search key, pointer>`

Search key: attribute(s) used to look up records

Pointer: where the matching data is

Why is an index file much smaller than the table?

Each entry stores only the key and a pointer, not the whole row. Many more entries fit in one block, so the index needs far fewer blocks.

Worked example: a 300,000-record file

Given

r — **300,000** records

B — **4,096** bytes per block

R — **100** bytes per record (fixed, unspanned)

Derive

Blocking factor

$bfr = \lfloor B / R \rfloor = \lfloor 4{,}096 / 100 \rfloor$

40 records per block

Number of blocks

$b = \lceil r / bfr \rceil = \lceil 300{,}000 / 40 \rceil$

7,500 blocks



… 7,500 blocks on disk

Cost to find one record

3,750

block accesses on average

No index, linear search on a unique key: b / 2 (worst case 7,500)

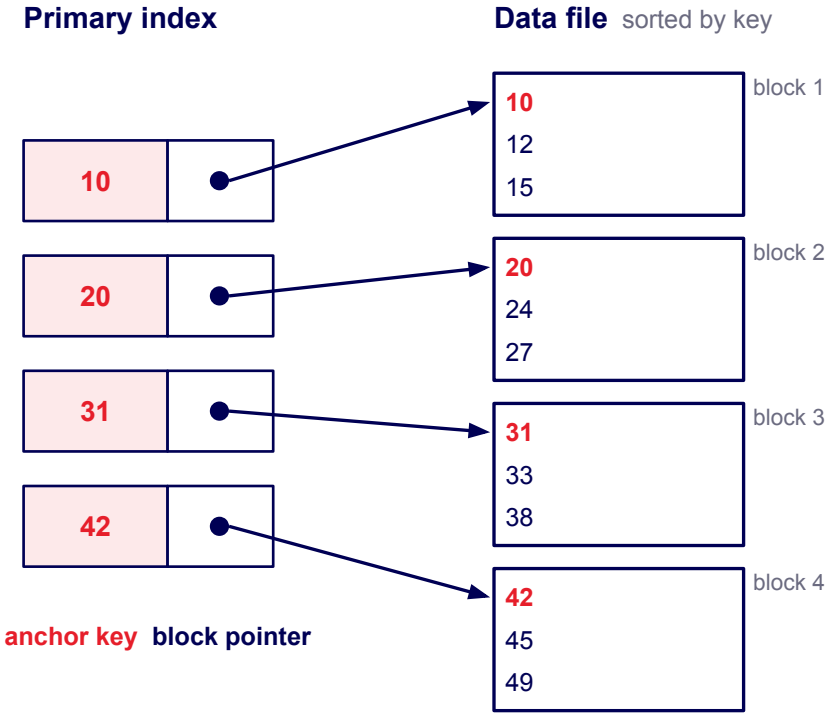
13

block accesses

File sorted on the key, binary search: $\lceil \log_2 7{,}500 \rceil$

Disk work is counted in blocks, the unit read from disk. InnoDB calls them pages (16 KB); this example uses 4 KB blocks to keep the numbers small.

Primary index: one entry per block (sparse)



- Built on an **ordered** data file, sorted on a key field.
- One entry **per block**: the key of the block's first record, the **block anchor**.
- So it is **sparse** (non-dense): far fewer entries than records.
- To search: find the last anchor ≤ the key, then read that one block.

Primary index: how many block accesses?

1 Entry size

$Ri = V + P$
$= 9 + 6$

15 bytes

2 Entries per index block

$\lfloor 4{,}096 / 15 \rfloor$

273

3 Index entries

one per data block

7,500

4 Index blocks

$\lceil 7{,}500 / 273 \rceil$

28

5 Search

$\lceil \log_2 28 \rceil + 1$ data block

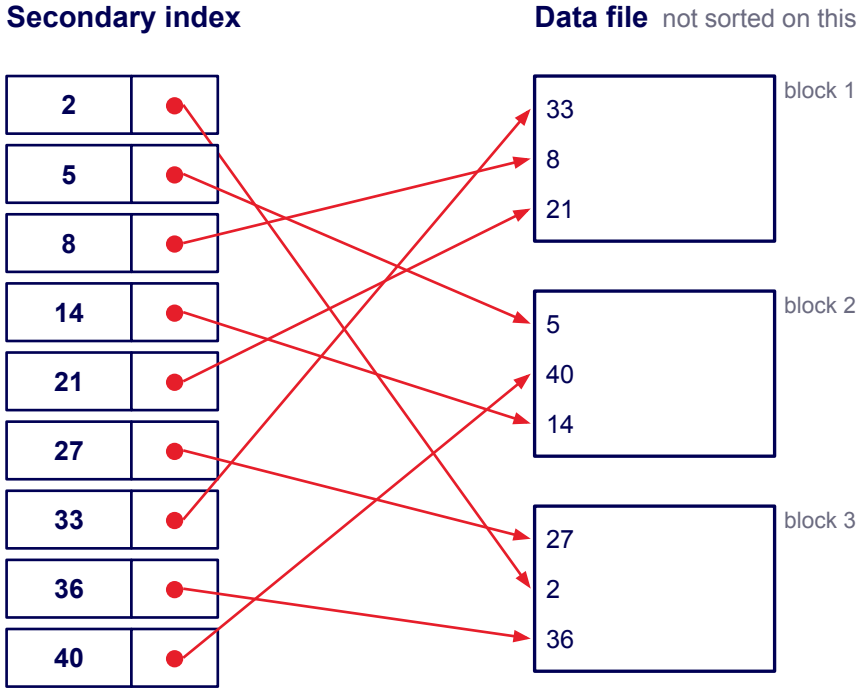
5 + 1

Key field V = 9 bytes, block pointer P = 6 bytes

13
binary search on the data file

6
with the primary index

Secondary index: one entry per record (dense)



- A **second way** to reach rows, when a primary access path already exists.
- Built on a **non-ordering** field of the data file.
- One entry **per record**, so it is **dense**.
- Pointer can be a **block pointer** (then search inside the block) or a **record pointer** (exact row).
- A table can have many: up to **64** secondary indexes in InnoDB.

Secondary index: how many block accesses?

1 Index entries

one per record

300,000

2 Entries per index block

$\lfloor 4{,}096 / 15 \rfloor$

273

3 Index blocks

$\lceil 300{,}000 / 273 \rceil$

1,099

4 Binary search

$\lceil \log_2 1{,}099 \rceil$

11

5 Plus the data block

11 + 1

12

Search on a UNIQUE key, V = 9 bytes, block pointer = 6 bytes

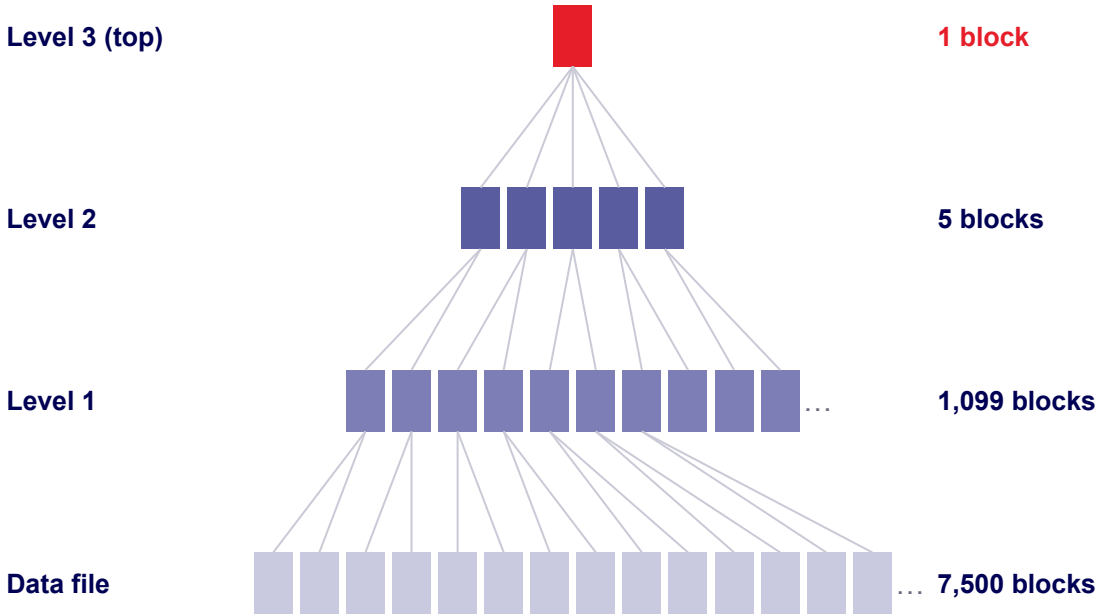
3,750

no index: linear search (file not sorted on this key)

12

with a secondary index

Multilevel index: an index on the index



Why must level 1 be bigger than one block? If it fits in one block, it already is the top level.

The rule

An index is itself an ordered file, so build a primary index on it. Repeat until the top level fits in one block.

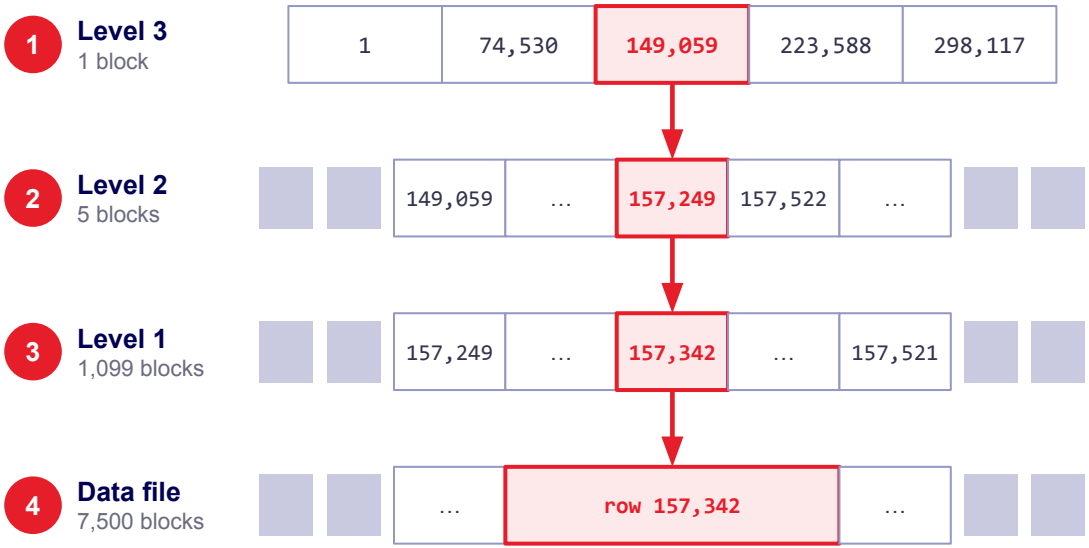
Our example

Fan-out = 273 entries per block. 1,099 → 5 → 1 block: 3 levels.

Cost = levels + 1 = **4 block accesses**

Multilevel search, step by step

Find key **157,342** among 300,000 unique keys (1 to 300,000). Fan-out = 273.

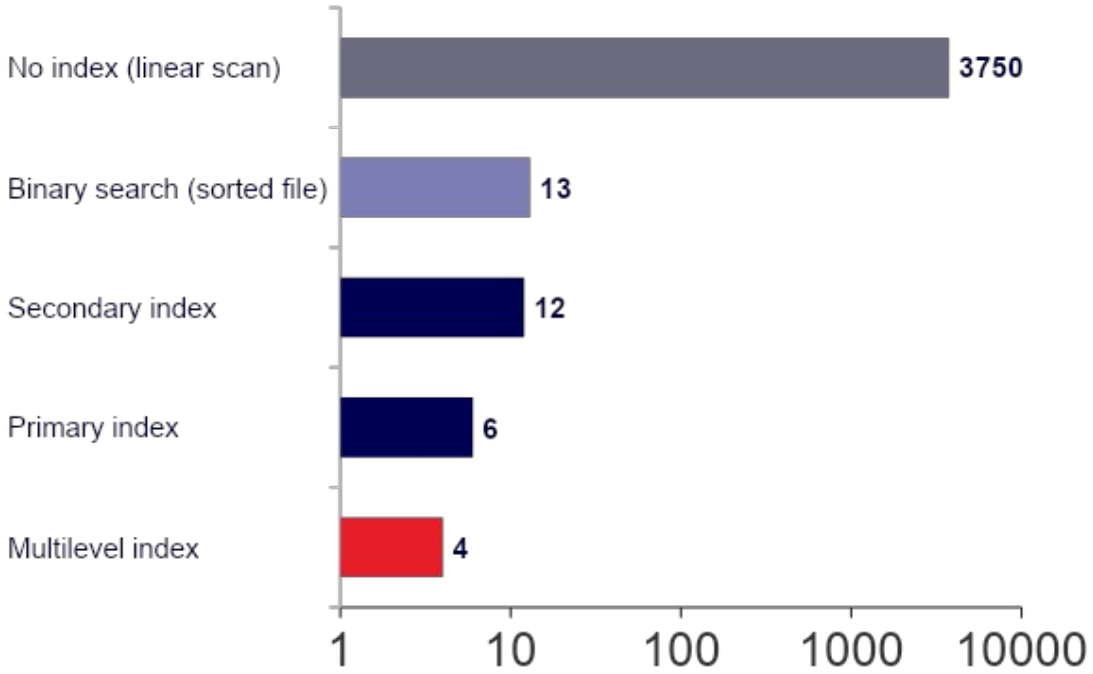


4 block reads

1. **Top block:** 149,059 ≤ 157,342 < 223,588 → level-2 block 3
2. **Level-2 block 3:** 157,249 ≤ 157,342 < 157,522 → level-1 block 577
3. **Level-1 block 577:** find 157,342 → pointer to its data block
4. **Data block:** read the row

Each read narrows the search 273× (the fan-out); binary search only halves it. Binary search on level 1: 11 + 1 = 12 reads. Multilevel: 3 + 1 = 4.

From thousands of reads to a handful



Log scale. Same 300,000-record file; values from the worked examples.

≈ 940× fewer

block reads: 3,750 → 4

Real databases use B+-trees: multilevel indexes that stay balanced as rows are added and removed. With big pages, 3 or 4 levels cover millions of rows.

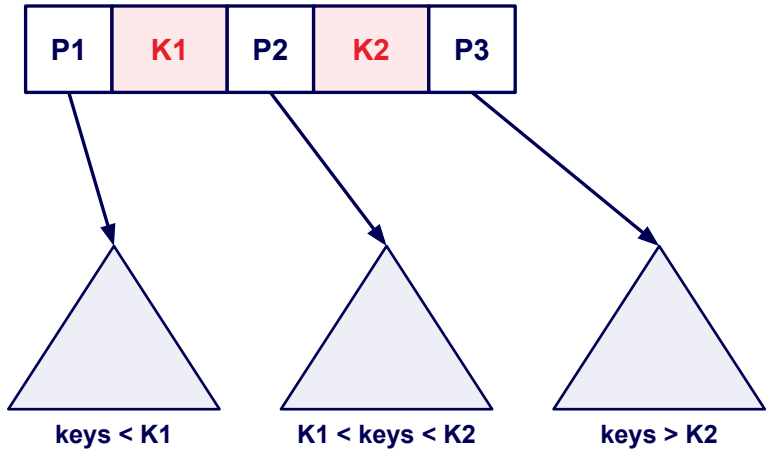
B-tree: a balanced multilevel index

A B-tree of order m

- Every node has at most m children
- Every internal node except the root has at least ⌈m/2⌉ children
- A node with k children holds exactly k − 1 keys
- All leaves are at the same depth (the tree is balanced)

In a database: each node is one disk block, kept between half-full and full. The spare room absorbs new entries; nodes split or merge to keep the tree balanced.

Inside one node



K = search key **P** = pointer to a child node (a disk block)

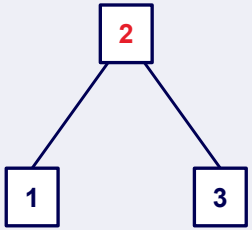
B-tree insertion, step by step

Each node has at most 3 children (2 keys). Insert 1, 2, 3, 4, 5, 6, 7.

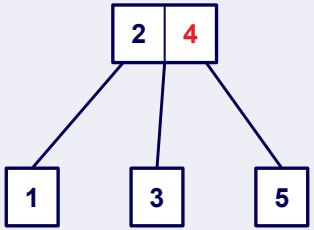
1 **Insert 1, 2**



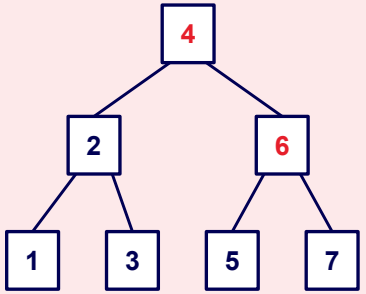
2 **Insert 3: leaf overflows, split**



3 **Insert 4, 5: split again**



4 **Insert 6, 7: root splits, tree grows**



Start at the root

Follow the keys down to a leaf

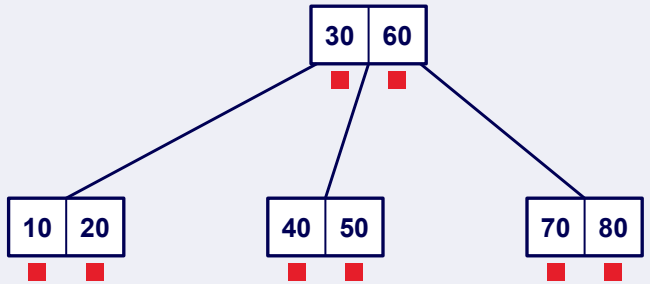
Room in the leaf? Insert. Done.

Full? Split around the median and push the median up. Repeat if the parent is full.

Red key = the median that moved up. Example after en.wikipedia.org/wiki/B-tree

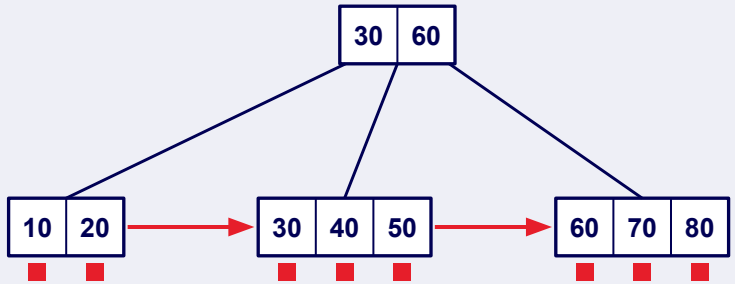
B-tree vs B+-tree

B-tree data pointers at every level



■ data pointer on every key

B+-tree data pointers only in the leaves



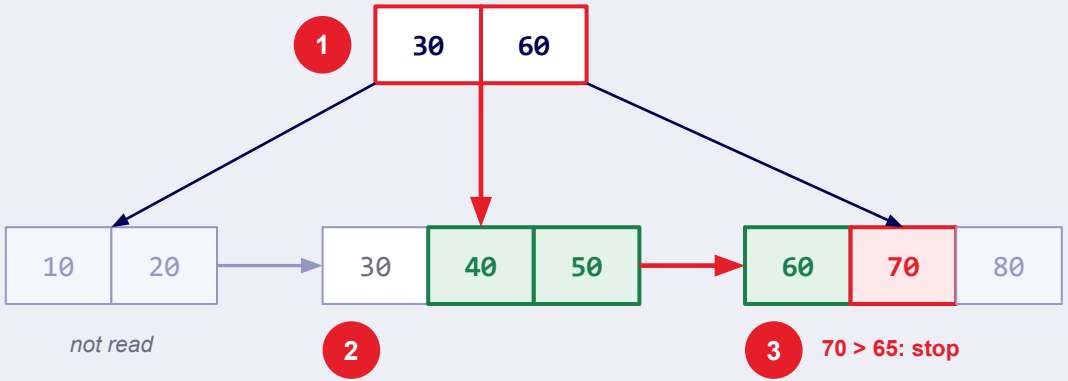
■ data pointer → next-leaf pointer

Why can a B+-tree have fewer levels?
Internal nodes hold only keys and child pointers, no data pointers, so more keys fit in each block: higher fan-out, fewer levels.

What are the leaf-to-leaf pointers for?
Range queries and ORDER BY: find the first key, then walk along the leaves in order without going back up the tree.

B+-tree range scan, step by step

`WHERE price BETWEEN 35 AND 65` on the B+-tree from the last slide



3 block reads

1. **Root:** 30 ≤ 35 < 60 → go to the middle leaf
2. **Leaf [30 40 50]:** skip 30, keep 40 and 50
3. **Next-leaf link → [60 70 80]:** keep 60; 70 > 65, so stop

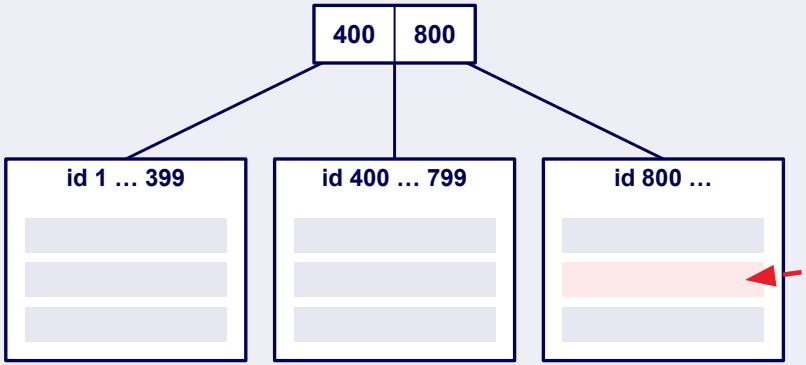
One descent, then a walk along the leaves. The keys come out in order, so ORDER BY price is free.

A plain B-tree has no leaf links: to reach the next key it must go back up through the parent, so range scans read more blocks.

How InnoDB uses B+-trees

Clustered index

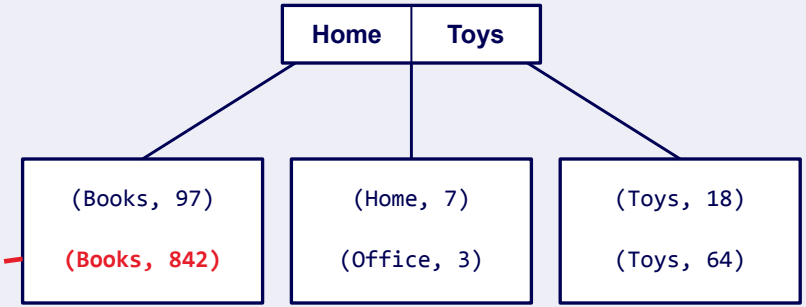
PRIMARY KEY (id): the table itself



leaves hold the full rows

Secondary index

INDEX (category)



leaves hold (key, primary key)

Lookup by category: search the secondary index, get the id, then search the clustered index for the row. Two tree searches.

Covering index: if the index holds every column the query needs, skip the second search. e.g. `COUNT(*) … WHERE category = 'Books'`

Index on multiple attributes (composite index)

`INDEX (city, age)`

city	age
Da Nang	21
Da Nang	35
Hanoi	19
Hanoi	22
Hanoi	40
HCMC	18
HCMC	25
HCMC	31

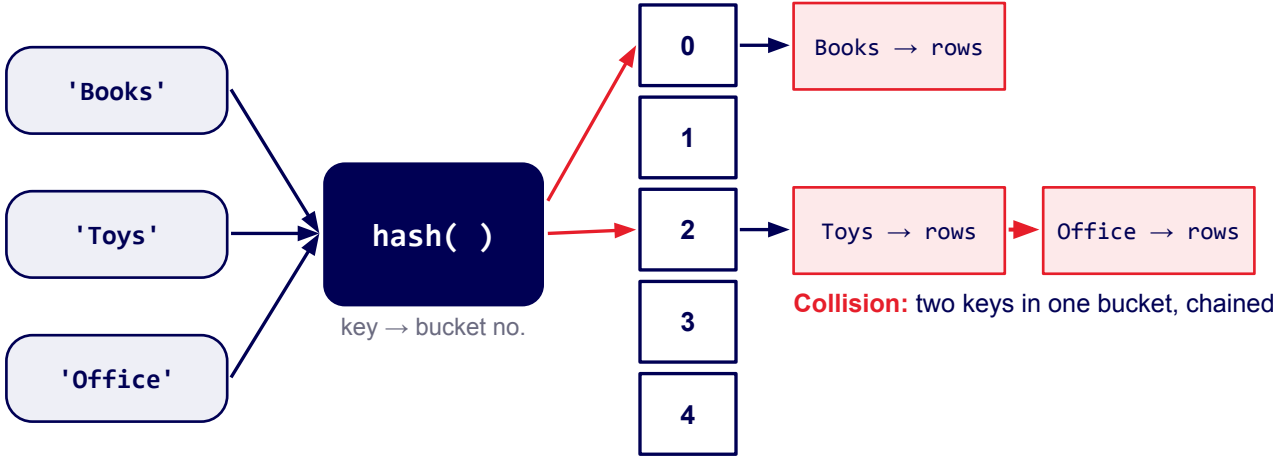
Sorted by city first, then by age within each city, like a phone book sorted by last name, then first name.

Which queries can seek with it?

Query		Why
WHERE city = 'Hanoi'	✔	leftmost column
WHERE city = 'Hanoi' AND age = 22	✔	both columns, in order
WHERE city = 'Hanoi' AND age > 20	✔	equality first, then range
WHERE age = 22	✘	age is not the leftmost column
ORDER BY city, age	✔	already in this order

Why not two separate indexes? If many rows match city = X and many match age = Y, but few match both, each single index still returns many rows. The composite index jumps straight to the few.

Hash index: a direct jump for equality



Good for

✔ = and <=> lookups, very fast

Not for

✘ ranges: <, >, BETWEEN

✘ ORDER BY

✘ part of a key: only whole keys

In MySQL: the MEMORY engine uses HASH indexes by default. InnoDB indexes are B+-trees: write USING HASH on an InnoDB table and you silently get a B-tree (SHOW INDEXES says BTREE). InnoDB's adaptive hash index is internal and automatic.

A B-tree can use any leftmost prefix of the key; a hash index cannot.



05

Using Indexes Well

Part 05

SQL syntax | EXPLAIN | Trade-offs | Guidelines | FULLTEXT & spatial

RMIT UNIVERSITY

Creating and managing indexes in SQL

Create an index

```sql
CREATE INDEX idx_category
  ON products (category);

CREATE INDEX idx_name
  ON people (last_name, first_name);
```

Define it with the table

```sql
CREATE TABLE products (
  id INT PRIMARY KEY,  -- indexed
  price DECIMAL(10,2),
  INDEX idx_price (price)
);
```

Add one with ALTER TABLE

```sql
ALTER TABLE products
  ADD INDEX idx_price_created
      (price, created_at);
```

Inspect and remove

```sql
SHOW INDEXES FROM products;

DROP INDEX idx_category
  ON products;
```

Column order matters: (last_name, first_name) and (first_name, last_name) are different indexes.

Is my index used? Ask EXPLAIN

```
EXPLAIN SELECT COUNT(*) FROM products WHERE category = 'Books';
```

	type	possible_keys	key	rows	Extra
Before	ALL	NULL	NULL	≈ 100,000	Using where
After	ref	idx_category	idx_category	≈ 12,500	Using index

Selected EXPLAIN columns; row counts are optimizer estimates and will differ on your data.

`type` ALL = full table scan · index = full index scan · ref / range = index lookup.

`key` the index actually chosen (NULL = none).

`rows` estimated rows to examine: smaller is better.

`Extra` "Using index" = covering: answered from the index alone.

Compare without dropping it: `… FROM products IGNORE INDEX (idx_category) WHERE …`

Week 3: EXPLAIN in depth

Indexes are a trade-off



Advantages

- Find rows without scanning the whole table
- Faster joins on indexed columns
- Faster ORDER BY and GROUP BY
- Enforce uniqueness (PRIMARY KEY, UNIQUE)
- Covering indexes answer queries without touching rows





Disadvantages

- Extra disk space for every index
- Slower INSERT, UPDATE and DELETE: each index must be updated too
- More indexes for the optimizer to consider
- Little benefit when a condition matches a large share of rows

When does an index help? Selectivity

Selectivity = how few rows a condition matches. The fewer, the more an index helps.

High selectivity: few rows — **Low selectivity: many rows**



`id = 123456`
1 row of 100,000
index helps a lot

`category = 'Books'`
≈ 1 in 8 rows
it depends

`price >= 0`
every row
a full scan is cheaper

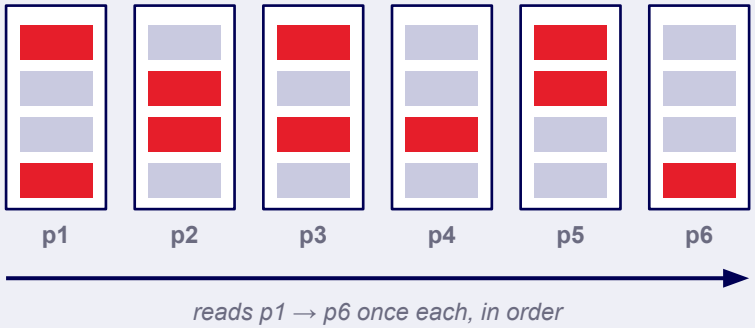
Why can a scan win? Following an index to many scattered rows costs more reads than reading the table straight through.

Exception: covering indexes. A COUNT(*) on a low-selectivity column can still be faster from a small index than from the full table.

Why a full scan can beat an index

`WHERE price < 400` matches about 40% of rows (10 of 24). 6 data pages × 4 rows; **red rows match**.

Full table scan



reads p1 → p6 once each, in order

6 page reads

sequential: the disk reads ahead

Through INDEX(price)

index, sorted

```
 12 → p3
 35 → p1
 58 → p5
 91 → p2
120 → p6
166 → p1
203 → p4
250 → p3
318 → p5
377 → p2
```

page reads, in index order



same page again (unless still cached)

10 page reads

random jumps, plus the index pages

Many matches spread over most pages → the scan wins. Few matches (high selectivity) → the index wins. The optimizer estimates both costs and picks the cheaper one.

Guideline 1: index for your retrieval queries

```
SELECT p.name, p.price          5
FROM products p                 1
JOIN order_items oi             1
  ON oi.product_id = p.id       4
WHERE p.category = 'Books'      2
  AND p.price BETWEEN 10 AND 50; 3
```

order_items is illustrative: it is not in the tutorial database.

For each important query, identify:

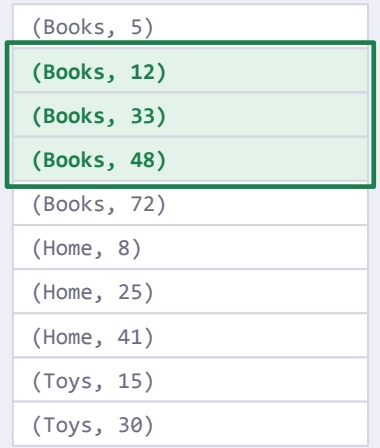
1. Which tables does the query access?
2. **Which attributes appear in selection (WHERE) conditions?**
3. Is each condition an equality, inequality or range?
4. **Which attributes link tables (JOIN conditions)?**
5. Which attributes are retrieved?

2 and 4 are your index candidates. 3 decides the column order of a composite index: equality columns first, then the range column.

Composite index: column order matters

```
WHERE category = 'Books' AND price BETWEEN 10 AND 50
```
equality first, then a range

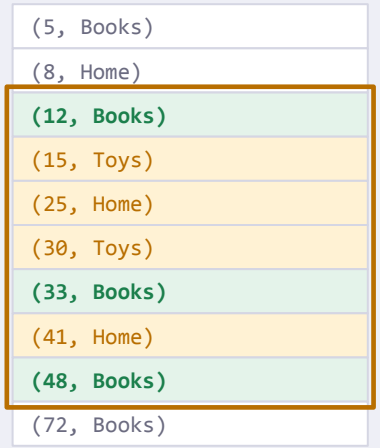
INDEX (category, price)



3 entries read, 3 match

Seek straight to (Books, 10), read until the price passes 50. The matches sit together: one short slice.

INDEX (price, category)



7 entries read, 3 match

The seek can only use price, a range. category is checked entry by entry, so it filters but cannot narrow the seek.

Rule: equality columns first, then the range column. After a range condition, later index columns can only filter, not seek.

Guideline 2: watch your modification queries

```
UPDATE products          1 2
SET stock = stock - 1      4
WHERE id = 42;             3
```

3 = index candidates

The database must find the rows to change, just like a SELECT.

Here id is the primary key, so it is already indexed. `DELETE ... WHERE category = 'Toys'` would benefit from INDEX(category).

For each modification, identify:

1. Which tables are updated?
2. Which operation: insert, update or delete?
3. **Which attributes are in the WHERE of the update or delete?**
4. **Which attributes does the update change?**

4 = think twice before indexing. Every change to an indexed column must also update the index, so frequently updated columns make writes slower.

FULLTEXT search in practice

Tutorial extra: import bbc-articles.csv, then search its **`text`** column.

```
CREATE FULLTEXT INDEX ft_text ON articles (`text`);

-- natural language mode, with a relevance score
SELECT category,
       MATCH(`text`) AGAINST ('Artificial Intelligence'
             IN NATURAL LANGUAGE MODE) AS score
FROM articles
ORDER BY score DESC
LIMIT 5;

-- boolean mode
SELECT * FROM articles
WHERE MATCH(`text`) AGAINST ('+Database MySQL' IN BOOLEAN MODE);
```

Boolean mode operators

`+word` must contain

`-word` must not contain

`word` optional: ranks higher if present

`'+Database MySQL'`
must contain Database; MySQL is optional but ranks higher

Quick check: answer in the chat

Q1

You have INDEX (city, age). Which condition can NOT seek with it?

1. `city = 'Hanoi'`
2. `age = 22`
3. `city = 'Hanoi' AND age = 22`

Q2

Best index type for WHERE price BETWEEN 100 AND 200?

1. **Hash**
2. **B+-tree**
3. **FULLTEXT**

Q3

7,500 data blocks, 273 index entries per block. How many blocks does the primary index need?

1. **28**
2. **273**
3. **7,500**

Key takeaways

Disk access dominates the cost

so count and minimise block reads

An index = sorted <key, pointer> entries

multilevel B+-trees cut thousands of reads to a handful

InnoDB: clustered PK + secondary indexes

composite indexes follow the leftmost-prefix rule

Indexes speed reads but cost writes and space

index WHERE and JOIN columns, then check with EXPLAIN

Next week · W3

Query optimization

WHERE YEAR(Bdate) = 1990
ignores your index on Bdate. Why?

Also: AND vs OR, join order, subqueries vs joins

Tutorial this week: build a 100,000-row playground

1

Create the database

```
CREATE DATABASE shopping;
USE shopping;
```

2

Create products

```
id, name, category,
price, stock, created_at
```

3

Insert sample data

```
CALL insert_sample_
  products();
```

4

Check the indexes

```
SHOW INDEXES
  FROM products;
```

What the generated data looks like

- 100,000 products (change the number in the WHILE loop)
- 8 categories: Electronics, Computer, Books, Clothing, Toys, Home, Grocery, Office
- price: random 0 to 1,000 · stock: 0 to 999
- created_at: a random day in the last 365 days

Before you index anything: SHOW INDEXES lists only PRIMARY (on id). Every other query starts as a full table scan.

Inserting 100,000 rows takes a while, so start it early.

Predict before you measure

For each tutorial query: will an index help, and which one? Write your guess, then test it.

#	Tutorial query	Think about…	Your prediction
1	Count products in category 'Books'	only 8 categories; covering index?	
2	Count price > 800 AND created in the last 100 days	one index or a composite one?	
3	Count price < 100 OR created in the last 50 days	can one index serve both sides of OR?	
4	Count categories containing the letter 'i'	LIKE '%i%' and the leftmost prefix	
5	The 100 most expensive products	ORDER BY price DESC LIMIT 100	

Then try FULLTEXT: find articles relevant to Artificial Intelligence in bbc-articles.

How to measure: with vs without an index

1. **Run it** — note the duration MySQL reports
2. **EXPLAIN it** — type, key, rows
3. **CREATE INDEX** — on the column(s) you chose
4. **Re-run + EXPLAIN** — did key change? fewer rows?
5. **Compare** — then IGNORE INDEX or DROP INDEX to reset

```
SELECT COUNT(*) FROM products WHERE category = 'Books';
EXPLAIN SELECT COUNT(*) FROM products WHERE category = 'Books';
CREATE INDEX idx_category ON products (category);
-- repeat the SELECT and the EXPLAIN, then compare
```

Run each query 2 or 3 times. The first run may read from disk; later runs often hit the buffer pool in memory.

Record a small table: query · time without index · time with index · index used (key).

Thank you

Questions?

RMIT UNIVERSITY



Copyright © 2023 RMIT University Vietnam